Name: Hifza Hussain

Roll no: 330

Section: BSAI-3A

Task 5: MINI MAX and Alpha beta

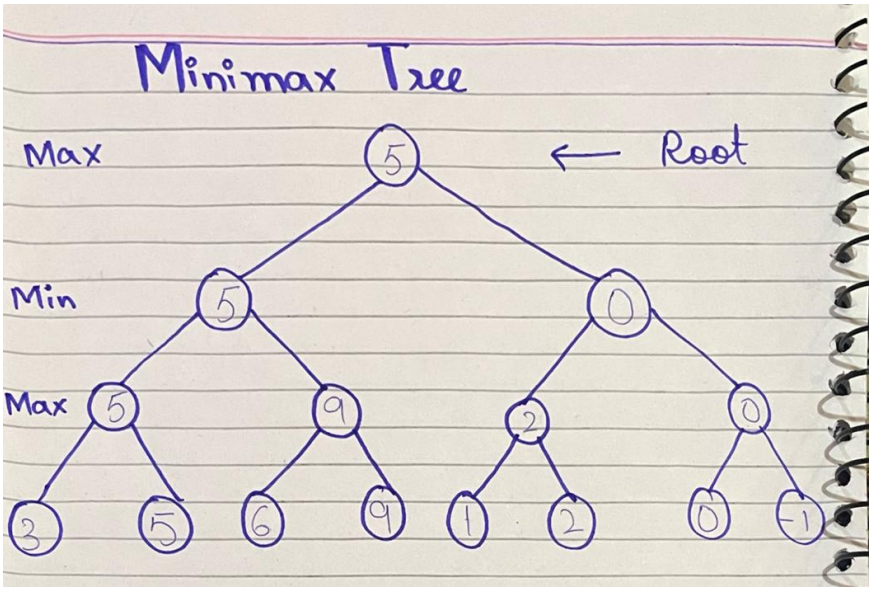
Game Tree:

```
tree = [
    [[3, 5], [6, 9]],
    [[1, 2], [0, -1]]
]
```

Tree structure:

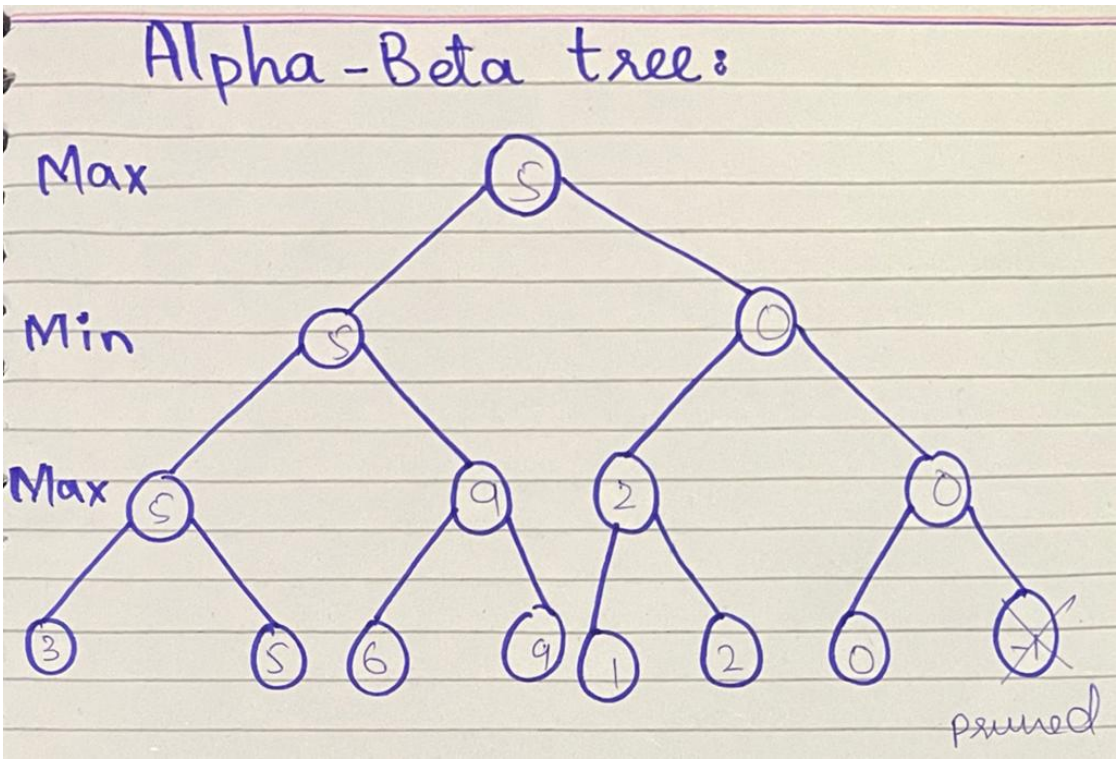
- Root = MAX
- Root ke children = MIN
- MIN ke children = MAX
- Last values = Leaf nodes

Mini Max Tree:



Node	calculation	value
Max 1	(3, 5)	5
Max 2	(6, 9)	9
Min 1	(5, 9)	5
Max 3	(1, 2)	2
Max 4	(0, -1/3)	0
Min 2	(2, 0)	0
Root max	(5, 0)	5

Alpha Beta tree:



Node	calculation	α	β	value
Max1	3,5	5	-	5
Max2	6,9	9	-	9
Min1	5,9	5	5	5
Max3	1,2	5	2	2
Max4	0	5	2	0
Pruning	-1 skip	5	2	-
Min2	2,0	5	0	0
RootMax	5,0	5	-	5

Python code:

minimax,alphabeta.py > ...

```python

tree = [
    [[3, 5], [6, 9]],
    [[1, 2], [0, -1]]
]

minimax_count = 0
alphabeta_count = 0


def minimax(node, maximizing):
    global minimax_count

    if isinstance(node, int):
        minimax_count += 1
        return node

    values = []

    for child in node:
        values.append(minimax(child, not maximizing))

    if maximizing:
        return max(values)
    else:
        return min(values)


def alpha_beta(node, alpha, beta, maximizing):
    global alphabeta_count

    if isinstance(node, int):
        alphabeta_count += 1
        return node

    if maximizing:
        value = float('-inf')

```

minimax,alphabeta.py > ...

```
29  def alpha_beta(node, alpha, beta, maximizing):
38
39          for child in node:
40              value = max(
41                  value,
42                  alpha_beta(child, alpha, beta, False)
43              )
44
45              alpha = max(alpha, value)
46
47              if alpha >= beta:
48                  break
49
50          return value
51
52      else:
53          value = float('inf')
54
55          for child in node:
56              value = min(
57                  value,
58                  alpha_beta(child, alpha, beta, True)
59              )
60
61              beta = min(beta, value)
62
63              if alpha >= beta:
64                  break
65
66          return value
67
68
69  minimax_count = 0
70  alphabeta_count = 0
71
```

minimax,alphabeta.py > ...

```python
minimax_value = minimax(tree, True)

alphabeta_value = alpha_beta(
    tree,
    float('-inf'),
    float('inf'),
    True
)

print("ORIGINAL TREE")
print("Minimax Value:", minimax_value)
print("Alpha-Beta Value:", alphabeta_value)
print("Minimax Leaves:", minimax_count)
print("Alpha-Beta Leaves:", alphabeta_count)

reordered_tree = [
    [[-1, 0], [2, 1]],
    [[9, 6], [5, 3]]
]

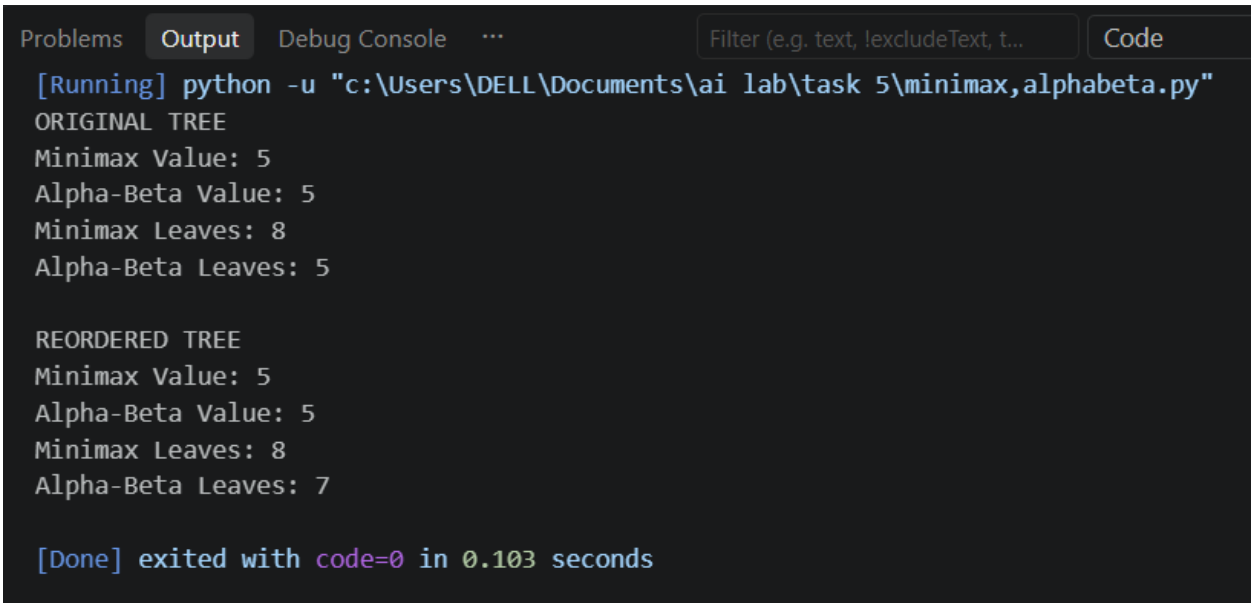
minimax_count = 0
alphabeta_count = 0

minimax_value2 = minimax(reordered_tree, True)

alphabeta_value2 = alpha_beta(
    reordered_tree,
    float('-inf'),
    float('inf'),
    True
)

print("\nREORDERED TREE")
print("Minimax Value:", minimax_value2)
print("Alpha-Beta Value:", alphabeta_value2)
print("Minimax Leaves:", minimax_count)
print("Alpha-Beta Leaves:", alphabeta_count)
```

Output:



Comparison Table:

Algorithm	**Root Value**	**Evaluated Leaves**
Minimax (Original)	5	8
Alpha-Beta (Original)	5	7
Minimax (Reordered)	5	8
Alpha-Beta (Reordered)	5	8

Poor Branch Ordering

When branches are arranged in an unfavorable order, Alpha-Beta Pruning may evaluate more leaf nodes and perform less pruning.

In our reordered tree, Alpha-Beta evaluates 8 leaves instead of 7. However, the root value remains 5.

This shows that Alpha-Beta Pruning efficiency depends on branch ordering. It does not always reduce the number of leaf evaluations.

Reflection:

In this lab, I implemented the Minimax and Alpha-Beta Pruning algorithms using Python. I learned how MAX and MIN nodes select values from a game tree. I also added leaf evaluation counters to compare both algorithms.

I observed that Alpha-Beta Pruning can reduce unnecessary leaf evaluations while maintaining the same optimal root value. I also learned that branch ordering affects pruning efficiency. Good ordering can improve performance, whereas poor ordering can reduce pruning.

Both algorithms provide the same optimal result for the same complete game tree.